Online Training
C# 6.NET Online Training Program
ASP.NET Core Training
Microservices Online Training Program
Dot Net Training
Microsoft Azure Training
Angular Training

Introduction & Environment
C# Introduction
Hello Program Windows Form Application
Introduction to Programming Languages
Hello Program Console Application
Different Types of Applications
Programming Methodologies
Algorithm, Pseudocode, Flowchart
Introduction to .NET Framework
Introduction to .NET Framework
.NET Framework Architecture and Components
Introduction to Object Oriented Programming Language
How to Download and Install Visual Studio in Windows
Creating First Console Application
Understanding Visual Studio IDE
.NET Developer Roadmap for 2020
Coding Standard Practices

C# .NET Basics
Data Types
Introduction of C# Program
Methods and Properties of Console Class in C#
Data Types in C#
Variables in C#
Variables in C#
Control Flow Statements in C#
If-Else Statements in C#
Nested If-Else Statements
Loops in C#
While Loop in C#
While Loop in C#
For Loop in C#
Nested For Loop in C#
Continue Statement in C#
Goto Statement in C#
Functions in C#
User-Defined Functions in C#
Call by Value and Call by Reference in C#
Recursion in C#
Pass Array as Function Parameter in C#
Command Line Arguments in C#
Strings in C#
Strings Continued in C#
Static vs Dynamic Functions in C#
Const and Read Only in C#
Const and Read Only in C#
Operators in C#
Operator Precedence and Associativity in C#
Operator Overloading in C#
Operator Overloading in C#
Method Overloading in C#
Method Hiding in C#
Partial Class and Partial Methods in C#
Sealed Class and Sealed Methods in C#
Extension Methods in C#
Static Class in C#
Variable Reference and Instance of a Class in C#

OOPs Real Time Example
Real-time Examples of Encapsulation Principle in C#
Real Time Examples of Abstraction Principle in C#
Real Time Examples of Inheritance Principle in C#
Real Time Examples of Polymorphism Principle in C#
Real Time Examples of Interface Principle in C#
Real Time Examples of Abstract Class in C#

Exception Handling
Exception Handling in C#
Multiple Catch Blocks in C#
Finally Block in C#
How to Create Custom Exceptions in C#
Inner Exception in C#
Exception Handling Issues in C#

Events, Delegates and Lambda Expression in C#
Delegates Introduction of Events, Delegates and Lambda Expression
Basics of Events, Delegates and Event Handler in C#
Multicast Delegate in C#
Multicast Delegates in C#
Delegates Real Time Example in C#
Generic Delegates in C#
Anonymous Methods in C#
Lambda Expressions in C#
Events in C# with Examples

Multi Threading
Multi Threading in C#
Thread class in C#
How to Pass Data to Thread Function in Type Safe Manner in C#
How to Retrieve Data from a Thread Function in C#
Join Method and Isalive Property of Thread Class in C#
Thread Synchronization in C#
Locks in C#
Monitor Class in C#
Pulse Class in C#
Semaphore Class in C#
SemaphoreSlim Class in C#
Mutex Class in C#
Performance Testing of a Multithreaded Application
Thread Pool in C#
Asynchronous Programming Models in C#
Thread Pool in C#
Thread Cancellation in C#
Thread Pulse in C#
Timer Thread Communication in C#
How to Sitting a Multi-threaded Application in C#

Collections in C#
Arrays in C#
Array Class in C#
Advantages and Disadvantages of Arrays in C#
Collections in C#
Arraylist in C#
Hashtable in C#
Non Generic Stack in C#
Non Generic Queue in C#
Non Generic Hashtable in C#
Advantages and Disadvantages of Non Generic Collection in C#
Generic Collections in C#
Generic List Collection Class in C#
Generic Stack in C#
Generic Queue in C#
Generic Dictionary in C#
Generic Collections in C#
Common Between Array List and Dictionary in C#
List vs Dictionary in C#
Generic Sorted List Collection Class in C#
Generic Sorted Dictionary Collection Class in C#
Generic SortedSet Collection Class in C#
Generic SortedDictionary Collection Class in C#
Generic LinkedList Collection Class in C#
Generic HashSet Collection in C#
ConcurrentDictionary Collection Class in C#
ConcurrentQueue Collection Class in C#
ConcurrentStack Collection Class in C#
ConcurrentBag Collection Class in C#
BlockingCollection in C#

File Handling
File Handling in C#
Directory Class in C#
DriveInfoExample1 in C#
DriveInfoDrive in C#
File Class in C#
TextWriter and TextReader in C#
StreamWriter and StreamReader in C#
BinaryWriter and BinaryReader in C#
Directory Class in C#
Directory Class in C#
Asynchronous Programming
Introduction to Concurrency
Async and Await in C#
Task in C#
How to Return a Value from a Task in C#
How to Execute Multiple Tasks in C#
How to Limit Number of Concurrent Tasks in C#
Task Cancellation in C# using Cancellation Token
How to Create Synchronous Method using Task in C#
Race Patterns in C#
Only One Pattern in C#
How to Control the Result of a Race in C#
Task Based Asynchronous Programming in C#
Chaining Tasks by using Continuation Tasks
Task Based Asynchronous Pattern Task in C#
IsTaskFaulted in C#
How to Cancel a Non-Cancellable Task in C#
Cancellation Token in C#
How to Cancel Asynchronous Stream in C#

Parallel Programming
Task Parallel Library in C#
Parallel for in C#
Parallel Foreach Loop in C#
Parallel Invoke in C#
Plinq in C#
Parallelism Degree of Parallelism in C#
How to Cancel Parallel Operations in C#
Atomic Methods, Thread Safety and Race Conditions in C#
Interlocked in Lock in C#
Parallel LINQ in C#
PLINQMapping in Asynchronous Programming -> Parallel Programming in C#

AutoMapper
AutoMapper Guide
AutoMapper Complex Mapping in C#
How to Map Complex Type to Primitive Type using AutoMapper in C#
AutoMapper Reverse Mapping in C#
AutoMapper Conditional Mapping in C#
AutoMapper Ignore Method in C#
Fixed and Dynamic Values in Destination Property in AutoMapper

Optional Parameter, Indexers and Enums
Optional Parameters Optional Parameters in C#
Indexers in C#
Indexers Real Time Example in C#
Enums in C#

.NET Framework Architecture
.NET Framework
Common Language Runtime (CLR) in C#
Common Type System in .NET Framework
Common Language Specification in .NET Framework
Managed and Unmanaged Code in .NET Framework
Assembly PE, CIL & .NET Framework
Assembly in .NET Framework
Strong and Weak Assemblies in .NET Framework
How to Install an Assembly in the GAC in .NET Framework
CIL, JIT Compilers and Solution in .NET Framework

Dos, Dynamic and Reflection
Reflection in C#
Dynamic Type in C#
Var Keyword in C#
Var vs Dynamic in C#
Dynamic vs Reflection in C#
Dynamic Keyword in C#
Dll vs Exe in C#
Named Parameters in C#

C# Less Features
C# 7 New Features
Enhancements in Out Variables in C# 7
Pattern Matching in C#
Digit Separators in C# 7
Ref Locals in C#
Defining Types in C# 7
Local Functions in C#
Ref Returns and Ref Locals in C#
Discards
Enhanced Expression Bodied Types in C# 7
Expression Bodied Members in C#
Throwing Expression in C#
Async Main in C#

C# 8 New Features
C# 8 New Features
Nullable Types in C#
Nullable Reference Methods in C#
Pattern Matching in C#
Using Declarations in C#
Static Local Functions in C#
Disposable Ref Structs in C#
Nullable Reference Types in C#
Asynchronous Streams in C#
Asynchronous Disposable in C#
Indices and Ranges in C#
Null Coalescing Assignment Operator in C#
Unmanaged Constructed Types in C#
Charactiles in C#

Most Popular C# Books
Pro C# 7: With .Net and .Net Core
Pro C# 7
Pro C# Microsoft Data Structures and Algorithms Books using C#

Common Type System in .NET Framework

Back to C#.NET Tutorials For Beginners and Professionals

Common Type System (CTS) in .NET Framework

In this article, I am going to discuss the **Common Type System in .NET Framework**. Please read our previous article, where we discussed the Intermediate Language (CLR)/CIL and JLS/JVM in .NET Framework and Common Type System (CTS) in .NET Framework before look into type system and declared, used and managed in the Common Language Runtime (CLR). At the end of this article, you will understand what is Common Type System (CTS) in C# and How CTS in .NET works.

What is the Common Type System in .NET Framework?

The .NET Framework supports many programming languages such as C#, VB.NET, J#, F#, etc. Every programming language has its own data type. One programming language data type cannot be understood by other programming language. But there can be situations when we need to communicate between two different programming languages. For example, we need to write code in the .NET language and that code may be called from C# language. In order to ensure smooth communication between different programming languages, Microsoft has introduced Common Type System (CTS) which ensures that data types defined in two different languages get compiled to a common data type.

CLR in .NET Framework will execute all programming languages data types. This is possible because CLR has its own data types which are common to all programming languages. At the time of compilation, all language-specific data types are converted into CTS data type. This data type system of CLR is common to all .Net Supported Programming languages and this is known as the Common Type System (CTS) in .NET Framework.

Example to understand Common Type System in .NET Framework

Let us understand Common Type System (CTS) in .NET Framework with an example. What we are going to do is, we will create two applications. One Application uses the C# Language and the other one is using VB.NET Language. And then we will try to see the C code of both of these applications and then we will try to see what the CTS looks like.

Here we are going to create two class library projects. One class library project uses the C#.NET language and the other class library project uses the VB.NET language.

Creating C# Class Library Project:

Create a class library project with the name **CsharpClassLibrary** and use the C# programming language. Once you create the C# class library project then add a class file with the name **CalculateJava** and then copy and paste the following code into it.

```
public class CalculateJava
{
    public int[] GetJavaArray()
    {
        int a = 5, b = 25;
        int[] c = a * b;
        return c;
    }
}
```

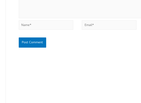
Creating VB Class Library Project:

To do same CalculateJava class, let us add another class library project with the name as **VBClassLibrary** and using **VB** as the programming language. Once you created the VB Class library project then add a class file with the name **CalculateJava** to the project and copy and paste the following code into it.

```
Public Class CalculateJava
    Public Function GetJavaArray() As Integer
        Dim a As Integer = 5
        Dim b As Integer = 10
        Dim c As Integer
        c = a * b
        Return c
    End Function
End Class
```

Now, build the solution which should generate the respective dll. In our previous article, we discussed how to use the **ILDASM** tool to view the IL code in detail. So, let us open Visual Studio Command Prompt in Administrative mode and run two instances of the **ILDASM.exe** command i.e. one for the VB class library project DLL file and the other one for the C# class library project DLL file as shown in the below image.

Now, let us open the IL code of the Calculate method of both the class library projects as shown in the below image. Please have a look at the integer variable in the IL code which is int32 in this case. In the C# class library project, we use int as the type and its decimal variable is 5, and it's stored in the IL code library project, we use integer as the data type to receive the variable is 5, and in the end of the day, both the data types are compiled to a common data type i.e. Int32.



So, whether we write the code in VB.NET or in C#.NET, it follows the dot net rules or specifications, at the end of the day it is compiled into a Common Type System (CTS) .NET Framework which is common for all .NET Supported Programming Languages. This Common Type System is Provided to CLR and is the same for all .NET Supported programming languages. While compiling the source code into the executable file, the compiler, they will convert the language-specific data type to CLR-specific data type. For example, in C# and Integer in VB is converted to Int32 after compilation and Int32 is the data type which is understood by CLR. So you can see Int32 is the CLR-specific data type.

We already demonstrated IL code and CTS in C#. So, in the next article, I am going to demonstrate to you the example of Common Language Specification (CLS) in detail. Here, in this article, I try to explain the **Common Type System (CTS) in .NET Framework** in detail and I hope you enjoy CTS (Common Type System) in the C#.NET Framework article.

Dot Net Tutorials
About the Author: Pranaya Rout
Pranaya Rout has published more than 3,000 articles in his 12 year career. Pranaya Rout has very good experience with Microsoft Technologies, including C#, ASP.NET MVC, ASP.NET Web API, ASP.NET Web MVC, EF Core, ADO.NET, LINQ, SQL Server, Azure, Angular, Bootstrap, Ajax, Jquery, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Previous Lesson
Intermediate Language (CLR)/JVM & JLS/JVM Code in C#

Next Lesson
Common Language Specification in .NET Framework

4 thoughts on "Common Type System in .NET Framework"

ANM
AUGUST 17, 2021 AT 10:20 PM
Thank you for this.
I was only able to create the IL for the VBClassLibrary. I couldn't do the same for the CsharpClassLibrary and also for the CTS program in the previous article.
Reply

AMBACHANDAN P
SEPTEMBER 20, 2021 AT 10:10 PM
Sir Ma,
I have a doubt, sir, in the IL itself the we have the common datatype (Int32). while language compiler compile source code into MSIL. But in the Base Class Library (BCL) we have common data type the CTS is a set of common data type. Why we actually need to connect to CTS, so how MSIL have a common data type for all language ? I have this doubt for sure,Sorry please help me to come out.
Reply

DOT NET TUTORIALS
MAY 9, 2022 AT 2:07 PM
The Language Compiler while generating the IL Code will follow the CTS and CLS of CLR.
Reply

MANISH
MARCH 4, 2022 AT 12:49 AM
The previous C# code is performing a simple addition of two numbers values via the Calculation method. The .NET libraries do not contain platform-specific instructions but rather use an approach, IL code that is generated using the corresponding CIL compiler (i.e.csc) during the build process.
Managed code is executed by the CLR which manages that .NET library into memory and you end up with a single .Net assembly (dll) containing a manifest, metadata, and IL instructions (instead of machine code). The Framework ships with an excellent utility to disassemble any .NET binary into its corresponding IL code referred to as "ILDASM.EXE".
Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name * Email * Website

Post Comment